



REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

Producto VI: Liderazgo, Equipos y Cultura de Ingeniería

Profesor:

Mg. Félix Márquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Ciudad Guayana, septiembre de 2026

Introducción

En el ámbito de la ingeniería de software y la gestión del talento tecnológico, el rendimiento de una organización de tecnología no depende de manera aislada de sus herramientas o de la pila técnica elegida, sino de la alineación sistémica entre su arquitectura de software, su estructura organizacional y la salud interpersonal de sus equipos. Históricamente, la división del trabajo basada en silos funcionales o especializaciones técnicas rígidas ha obstaculizado la agilidad organizativa, generando dependencias cruzadas que incrementan los tiempos de ciclo, provocan demoras en las entregas y desgastan la dinámica de trabajo.

La interacción entre la estructura de una organización y la arquitectura del sistema que esta construye está gobernada por la Ley de Conway, la cual postula que los canales de comunicación interna dictan de forma inevitable los límites y el acoplamiento del software resultante. Para superar estas limitaciones y posibilitar evoluciones complejas como la migración a microservicios, las organizaciones deben recurrir a la Maniobra Inversa de Conway, reconfigurando sus equipos en torno al flujo de valor mediante modelos modernos como Team Topologies.

Respuesta 1: La Maniobra Inversa de Conway y Team Topologies

1. Diagnóstico de la Estructura Actual mediante la Ley de Conway

La Ley de Conway establece que las organizaciones encargadas de diseñar sistemas están restringidas a producir diseños que son copias exactas de sus propias estructuras de comunicación internas. En el contexto del banco, la separación departamental en silos puramente tecnológicos (Base de Datos, Backend y UI) fuerza la consolidación de una arquitectura monolítica altamente acoplada. La agilidad fracasa en esta estructura por las siguientes razones:

Fricción en el flujo de valor: Cada departamento optimiza metas locales y aisladas (el equipo de base de datos busca estabilidad de esquemas, el backend velocidad de cómputo y la UI diseño visual). Al estructurar los equipos en torno a capas técnicas en lugar de áreas funcionales del negocio, el lanzamiento de una característica como "Créditos" exige una comunicación de alto ancho de banda y una coordinación intensiva entre múltiples departamentos.

Pared de confusión y fricción de comunicación: La necesidad operativa de un "Gerente de Proyecto" tradicional para orquestrar la integración demuestra un defecto en el diseño organizacional generando una "pared de confusión", con trasposos (handoffs) secuenciales, demoras de meses y disputas de prioridad entre áreas.

2. Rediseño Organizacional con la Maniobra Inversa de Conway y Team Topologies

Para resolver este estancamiento técnico, se aplica la "Maniobra Inversa de Conway", un principio que plantea reestructurar los equipos y sus fronteras de comunicación para forzar que la arquitectura del software evolucione de manera natural hacia el modelo desacoplado deseado.

Utilizando los 4 tipos de equipos de Team Topologies, la estructura se conforma de la siguiente manera:

Stream-Aligned Teams (Equipos alineados al flujo de valor): Equipos multifuncionales que contienen todas las competencias requeridas (desarrolladores UI, backend Java, ingenieros de datos y QA) que integran todas las competencias necesarias para diseñar, desarrollar, probar, desplegar y operar el sistema de principio a fin. Al asignárseles un dominio específico (por ejemplo, el dominio integral de "Créditos"), estos equipos obtienen la libertad y la autoridad para modificar cualquier componente del código sin depender de áreas externas.

Platform Teams (Equipos de Plataforma): Su función es habilitar el trabajo de los equipos alineados al flujo proporcionando herramientas, infraestructura y entornos estandarizados de autoservicio (CI/CD, plantillas de microservicios, entornos de nube) que permiten a los Stream-Aligned Teams probar y desplegar de forma autónoma sin depender de aprobaciones manuales externas.

Enabling Teams (Equipos de Habilitación / Coaches): Especialistas en áreas avanzadas (como arquitectura de microservicios, seguridad o automatización) que capacitan y apoyan temporalmente a los equipos alineados al flujo para desarrollar nuevas capacidades, evitando convertirse en cuellos de botella permanentes, promoviendo un clima de aprendizaje continuo en la organización.

Complicated-Subsystem Teams (Equipos de Subsistema Complicado): Se hacen cargo de componentes de extrema complejidad técnica o matemática que requieren especialización profunda (por ejemplo, el motor del algoritmo crediticio en tiempo real o el módulo de cifrado bancario).

3. Estructura Organizacional para la Migración a Microservicios

Esta conformación establece límites de dominio claramente definidos que coinciden con las fronteras de los equipos. Cada Stream-Aligned Team es dueño de su propio microservicio y de su almacenamiento de datos. Esto minimiza la necesidad de coordinación intensa entre equipos, permitiendo realizar pruebas y despliegues independientes por demanda.

El objetivo primordial de esta arquitectura es que los equipos logren realizar cambios a gran escala, probar componentes de manera aislada (mediante técnicas de virtualización o dobles de prueba) y desplegar servicios a demanda sin requerir altos niveles de comunicación o coordinación con individuos externos a su equipo

4. Evolución del Antiguo "Gerente de Proyecto"

En este modelo, el rol de coordinación por comando y control desaparece al eliminarse los traspasos entre silos. El antiguo Gerente de Proyecto evoluciona hacia dos roles estratégicos dentro de la cultura ágil:

Product Owner / Product Manager: Asume la responsabilidad del backlog del producto, enfocándose en maximizar el valor del flujo de trabajo, definir prioridades y conectar la estrategia del negocio con el equipo.

Delivery Lead / Agile Coach: Se enfoca en remover impedimentos del entorno, optimizar el rendimiento del equipo, mejorar el flujo y actuar como un líder facilitador.

Respuesta 2: Liderazgo, Conflicto y Seguridad Psicológica

1. Diagnóstico: Seguridad Psicológica

La seguridad psicológica es el entorno que permite a los individuos discutir abiertamente los problemas y obstáculos sin temor a daños o represalias.

La falta de propuestas por parte de los desarrolladores junior indica una ruptura de esta seguridad, siendo la dinámica del equipo y la seguridad psicológica los aspectos más importantes para comprender el rendimiento general del grupo.

Las críticas sarcásticas del Arquitecto violan el principio fundamental de que las revisiones de trabajo deben formularse de una manera constructiva y no personal.

Impacto a largo plazo en el rendimiento:

Supresión de la innovación y del flujo de información: Al ser humillados, los desarrolladores junior dejan de proponer soluciones y optan por ocultar fallos o dudas. Un entorno de trabajo que carece de respeto y apoyo ignora el bienestar del equipo y contribuye directamente al agotamiento de los empleados (burnout), la deserción y la pérdida de productividad.

Caída de la velocidad y aumento del retrabajo: Sin una atmósfera cooperativa, honesta y enfocada, el equipo pierde cohesión y el rendimiento sostenible se degrada. La falta de comunicación transparente genera errores no detectados en etapas tempranas, disparando el tiempo de resolución y degradando el desempeño del equipo.

Desgaste profesional: La cultura del miedo. El miedo a las represalias técnicas impide asumir riesgos compartidos, lo que paraliza la experimentación y reduce la tasa en la que el equipo puede validar nuevas ideas, destruyendo el compromiso y reduce el Employee Net Promoter Score (eNPS) del departamento.

2. Estrategia de Intervención: Liderazgo de Servicio

El liderazgo de servicio requiere centrarse en el desarrollo y en las necesidades de los seguidores.

La intervención debe alejarse del mando y control tradicional para adoptar un modelo de líderes como entrenadores (coaches).

Bajo esta mentalidad, se debe fomentar la idea de que el trabajo de todos consiste en tres pilares: hacer el trabajo, mejorar el trabajo y desarrollar a las personas.

Desde la perspectiva del Liderazgo de Servicio, se reconoce el talento del arquitecto, pero se redefine su éxito: su valor ya no se mide por señalar errores con dureza, sino por su capacidad de potenciar y desarrollar las habilidades de sus compañeros.

3. Reestructuración del Proceso de Revisión de Código (Code Review)

Para objetivar el feedback y eliminar los sesgos personales, se aplican las siguientes medidas:

Automatización del feedback de formato y estilo: Configurar linters y herramientas de análisis estático en el pipeline de CI/CD para que la máquina valide automáticamente reglas de estilo, convenciones y patrones básicos antes de la revisión humana.

Guía Colectiva de Revisión de Código: Definir acuerdos claros donde el feedback deba ser siempre respetuoso, impersonal y orientado al código (por ejemplo, fundamentado en la legibilidad, mantenibilidad o pruebas), prohibiendo descalificaciones sobre el autor.

Promover revisiones entre pares (Peer Reviews) y Pair Programming: Fomentar sesiones de programación en pareja para que la transferencia de conocimiento ocurra en tiempo real y de forma colaborativa.

4. Dinámica para la Retrospectiva: Reconstrucción de la Confianza

Para restablecer la seguridad psicológica en la siguiente retrospectiva, se propone la dinámica "Acuerdos de Trabajo y Directiva Primaria":

Todo el equipo debe reunirse de forma regular para reflexionar sobre el proceso de entrega, analizando qué ha salido bien, qué ha salido mal y discutiendo ideas para mejorar.

Estructure la sesión utilizando el ciclo de Deming: planificar, hacer, estudiar/verificar, actuar.

Para reconstruir la confianza y el sentido de propiedad, asigne a un miembro del equipo la responsabilidad de apropiarse de cada idea de mejora generada.

En la siguiente reunión, esta persona deberá informar al resto del grupo sobre los resultados y las acciones tomadas.

Conclusión

La transformación de una organización tecnológica requiere comprender que la arquitectura del software y la estructura social de la empresa son dos caras de una misma realidad socio-técnica. El análisis realizado evidencia que la eliminación de los cuellos de botella en entornos operativos complejos exige la aplicación de la Maniobra Inversa de Conway, reemplazando las divisiones por capas técnicas aisladas por equipos multifuncionales alineados al flujo de valor (Stream-Aligned Teams). Esta estructura se sostiene gracias a plataformas autoservicio (Platform Teams) y habilitadores de capacidades (Enabling y Complicated-Subsystem Teams), lo que reduce drásticamente la sobrecarga de coordinación entre áreas y prepara el terreno para despliegues independientes mediante microservicios.

Asimismo, el rendimiento técnico sostenible es inseparable de la cultura humana y del estilo de liderazgo. Las conductas destructivas en los procesos de revisión de código o la falta de empatía debilitan la seguridad psicológica, impulsando al equipo hacia culturas burocráticas o patológicas orientadas al miedo, al ocultamiento de errores y a la pérdida de innovación. La transición hacia un liderazgo de servicio, acompañada de la automatización de métricas de calidad y la creación de acuerdos de convivencia transparentes en las retrospectivas, permite restaurar la confianza interpersonal, reducir el burnout y maximizar la entrega continua de valor al negocio.

Referencias Bibliográficas

- Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.). Addison-Wesley Professional.
- Driessen, V. (2010, 19 de enero). *A successful Git branching model*. nvie.com.
<https://nvie.com/posts/a-successful-git-branching-model/>
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations*. IT Revolution Press.
- Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley Professional.
- IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)* (3.^a ed.). IEEE Computer Society Press.
- Reestructuración Organizacional y Ley de Conway. (2024). *Documento de análisis y transcripción interna sobre arquitectura y organización socio-técnica*.